

構造持続理論は Lean で何を保証するか

— 証明書つき入力から境界主張を検証する —

Akihito Sunagawa

2026年6月5日

要旨

構造持続理論は、構造系の持続・崩壊・回復を雑に語らないための、構造維持における可能性と不可能性の境界を形式化する理論である。各ドメインで観測された測定や代理指標を、構造側の制約 L/B と有効資源 M という二つの読み取り座標へ整理し、どの条件で維持でき、どの条件で維持できないかを型付き境界インターフェースとして読む。現実を直接測るのではなく、与えられた入力からどの境界主張を認可できるかを固定する層である。

出発点は、各ドメインで個別に語られてきた失敗の原因を、資源側と構造側に分けて読むことである。資源側では、対象を維持する有効資源 M が条件付きの有効資源として足りているかを見る。名目資源 R は、その源にはなりうるが、 M の本体ではない。Lean 側では、 R をそのまま M へ潰すことを防ぐ境界として扱う。構造側では、対象がある機能や同一性 F を保つための構造 K 、維持可能領域 V_K 、その縮小として読まれる構造損失または純負担 L/B を見る。

この分けかたを短い読みとして書くと、構造側の制約・損失を L が読み、いまま実際に使える有効資源を M が読み、その有効資源に構造損失を反映した境界ポテンシャルを $S = M \exp(-L)$ として読む。ただし有効資源 M は単なる名目資源 R ではなく、その対象の維持に実際に使えるぶんだけを指す。 $S = 0$ は、その対象の維持がもう成り立たない崩壊境界である。

この Lean アーティファクトの中心は、その L/B と M の分離を各層で混同しないように固定することである。レジリエンスのような派生量も、普遍的な一枚スコアではなく、維持対象・想定される衝撃・期限・目標を固定したうえでの衝撃後の読み出しとして、同じ境界読み取りに載せる（詳細は本文）。

この切り分けにより、同じ「崩壊」に見える状態を、資源不足、構造損失、機能停止、回復不能、集合的同時回復不能、修復外部性として区別できる。Lean 形式化の役割は、この分解を実装の中で崩さず、明示された根拠から出る境界帰結を検証することである。

本稿の貢献は、持続・崩壊・回復主張を、構造損失 L/B と有効資源 M の二軸に分解し、Lean 上で証明済み・証明書要求・定義台帳・経験仮説を混同しない監査形式として実装したことである。最近の workload 境界層では、この監

査形式の上に、仕様固定された有効資源境界も現れ始めている。有限 stochastic workload law と有限候補有効資源集合から generated required support を構成し、covered side では risk control、shortfall side では scoped protocol obstruction と positive D を読む。さらに grid / bracket / refinement / bracket-limit 層、riskFrontier 生成層、positive-rate envelope、resource-conserving protocol class も同じ監査面に接続されている。

本稿では、防御的な断りを各所に散らさない。Lean が保証する範囲と保証しない範囲は第7節に集約し、本文では主張の強度を次の棚で読む。

証明済み定理

Lean 上で宣言され、監査表に分類された境界帰結。

証明書要求主張

型は用意されているが、使うには明示的な witness / bound / composition rule / protocol が必要な claim。

定義・台帳

データを束ねる、射影する、監査可能な形に整えるための宣言。

解釈・経験仮説

ドメイン側の検証、proxy 妥当性、介入予測、外部再現を待つ読み。

この形式化は、理論が大きくなったときにも、どの部分が証明済みで、どの部分が証明書要求で、どの部分が経験的研究プログラムなのかを説明し続けるための仕組みである。

1 形式化の二枝構造

形式化が守るのは、直列の因果鎖ではなく、構造側と有効資源側を分けて境界 readout へ合流させる二枝構造である。

構造側: $F, K \rightarrow (V_K, m) \rightarrow (L, B),$

有効資源側: claim scope, horizon, assumptions, path, certificates + optional $R \rightarrow M,$

境界 readout: $(L/B, M) \rightarrow S.$

F は維持したい機能や同一性、 K はそれを担う構造である。 V_K は維持可能領域、 m はその測度である。 L/B は構造側の損失または修復込み純負担である。 M は有効資源として license された temporal resource claim であり、 m や L から

生成される後段の量ではない。 R はその optional nominal source / anti-collapse boundary である。 S は構造損失を反映した境界ポテンシャルである。

この二枝構造があることで、形式化は「壊れた」という一語に戻らない。どの構造が、どの機能を、どの余地で維持しているのか。どの損失が構造側にあり、どの support claim が資格づけられているのか。どの読みが近似で、どの読みが証明済みか。Lean 側では、これらを別のデータと別の定理として扱う。

回復に必要な境界量は、この二枝構造の中心ではない。それは、特定の目標へ戻せるかを判定するときの追加レイヤーである。Lean 側でも、この境界量を構造オントロジーの中心へ移さないことが重要な設計方針になっている。

2 層を分ける

Lean 形式化では、次の層を分けている。

- Admission
- > Handoff
- > Ontology
- > Adapters
- > RuntimeFailure
- > Interventions
- > Proxy / Validation
- > Diagnostics / Reports

それぞれの役割は次の通りである。

Admission は、ドメインが形式読み取りへ入る前のチェックである。これはまだ構造持続対象そのものではない。

Handoff は、入場条件と構造オントロジーの根拠を、ドメイン側が両方出したものだけを束ねる。

Ontology は、構造側の $F/K/V_K/m/L/B$ と、有効資源側の qualified M claim、およびそれらを合流させた S readout を持つ対象を定義する。 R は M claim の optional nominal source / anti-collapse boundary として扱う。

Adapters は、その対象を既存の M/L 読み取りコアへ接続する。

RuntimeFailure は、実行時に証明された失敗を扱う。報告ラベルとは分ける。

Interventions は、介入がどの軸を動かすのかを分類する。

Proxy / Validation は、近似測定を候補、検証済み、公式などの状態に分ける。

Diagnostics / Reports は、構造側と資源側の読みを報告可能な形に束ねる。

大事なのは、上の層から下の層が自動で出ないことである。たとえば、Admission を通ったからといって、Ontology が自動的に正しいことにはならない。ドメインは別途、構造、維持可能領域、測度、資源、根拠を供給する必要がある。

この層分けにより、入場前の不足、構造持続対象としての整合性、実行時の失敗、近似測定、回復判定、外部性を同じものとして扱わないようにする。Lean は、理論を大きく見せるためではなく、この分解を実装の中で崩さないために使われる。

3 Lean が固定する読み取りピース

Lean で確認している基本的な核は、境界読み取りの各ピースを混ぜないためのものである。

構造損失の読みについては、比率だけに依存し、連続する縮小で加法的に合成され、正則性を満たすという指定された表現公理クラスの中で、対数型の損失が強制される。

正の質量軌道があり、損失を

$$d_t = -\log \frac{m_{t+1}}{m_t}$$

と読むなら、累積損失

$$L_n = \sum_{t < n} d_t$$

に対して

$$m_n = m_0 \exp(-L_n)$$

が成り立つ。

修復を含めるなら、損失 d_t と修復 r_t から

$$b_t = d_t - r_t$$

を置き、累積純負担 B_n に対して

$$m_n = m_0 \exp(-B_n)$$

が成り立つ。

さらに、有効資源 M と構造保持率を分けて読むと、

$$S = M \exp(-L)$$

または

$$S = M \exp(-B)$$

が得られる。構造因子が正なら、 S の非正境界は資源側の非正境界と一致する。

ここでの M は、temporal resource layer では raw stock ではなく、現在の target、horizon、assumptions、path dependencies、certificate に照らした射影量 M_{proj} として読む。Lean 側では、まず TemporalResourceProjectionReport が current/restated projection に入る claim と除外される claim を分ける。さらに ComputedTemporalResourceProjectionReport は、含まれた claim の amount の有限和として projectedEffectiveAmount を計算する。したがって、 $S = M_{\text{proj}} * \exp(-L_{\text{total}})$ の handoff では、 $\exp(-L_{\text{total}})$ が正であるため、 S readout の正性やゼロ境界は projected M 側に一致する。

この符号境界は、既存の fullPotential 系の符号定理と同じく、証明書つき読み取り層の中で、 M 側が非ゼロ性を握ることを確認するものである。

さらに StructuralPersistenceTemporalResourceSupportPlan は、projection sum の上で、support claim を必要有効資源量に届いたものとして扱ってよいかを監査する。形式化しているのは、feasible かつ compatible な support plan 全体を certified upper bound が上から抑え、その上限が required support を下回るなら、required support に届く feasible compatible plan は存在しない、という狭い obstruction である。

ここで出る結論は support threshold non-licensing である。そこから先の主張境界は第7節に集約する。

StructuralPersistenceRequiredSupportCertificate は、この required support threshold 自体の由来を標準化する。 M_{required} は裸の数値ではなく、target、horizon、claim、evidence qualification、boundary clearance、support-necessity obligation に scoped された supplied certificate から、狭い RequiredSupport へ射影される。この certificate は、support-claim context の threshold と一致するという明示的 handoff がある場合だけ、RequiredSupportNecessaryForClaim bridge を作れる。

この層で Lean が固定するのは、その threshold が target / horizon / claim から切り離された floating scalar として使われないことである。

さらに `StructuralPersistenceSupportClaimBound` は、この obstruction 側に対して attainability 側を対にする。すなわち、上界証明書が required support を下回る場合には declared support threshold は満たされず、逆に明示された feasible / compatible support plan が required support に届く場合には threshold satisfaction が得られる。この二つは、同じ `L_profile` totalization context のもとで扱われる。したがって、`L_profile` の合成則は依然として supplied composition-rule certificate を要求し、M 側の到達可能性も explicit plan witness を要求する。

`StructuralPersistenceQualifiedSupportFrontier` は、この上下の bound を一つの supplied exact finite boundary として束ねる。すなわち、ある qualified-support capacity が、全 feasible / compatible plan の上界であり、かつ明示 plan によって達成されるという certificate が与えられている場合に限って、declared support threshold は required $\leq$ capacity と同値になる。これは「限界定理」型の最小骨格である。

したがって、M 側の限界定理レイヤーは、現時点では **certified finite boundary** と読むのが正確である。supplied capacity certificate があるとき、不可能側と達成可能側を同じ C で挟む。

`StructuralPersistenceQualifiedSupportSearchCertificate` は、この C を完全に裸で与えるのではなく、best feasible / compatible plan と、全 feasible / compatible plan がその best value 以下であるという finite-search certificate から導く。`StructuralPersistenceQualifiedSupportNetworkFrontier` は、同じ形を finite network-path adapter として読む。

`StructuralPersistenceQualifiedSupportFrontierBridge` は、境界を domain claim predicate へ接続する最後の薄い bridge である。必要条件 bridge が supplied されていれば、capacity shortfall はその claim predicate を non-license できる。十分条件 bridge が supplied されていれば、threshold attainability はその claim predicate を license できる。

同じ層には、より薄い claim bridge として `RequiredSupportNecessaryForClaim` も置かれている。これは「ある domain claim を support layer で license するには、この support threshold が必要である」という supplied certificate である。この bridge が与えられているときに support threshold が満たされなければ、その claim は `ClaimLicensedUnderRequiredSupport` としては成立しない。逆に、explicit feasible / compatible plan が threshold に届く場合には、claim の support-threshold 側だけが license される。

required support がその domain claim の必要条件であること自体が supplied

obligation である。この interface は、どの条件で claim を non-license し、どの bundle があれば support threshold を attain したと言えるかを分けるための最小の bound interface である。

`StructuralPersistenceTemporalResourceSupportToy` は、この bound interface の有限 anti-collapse witness である。同じ raw support summary、同じ candidate plan set、同じ per-plan support value、同じ required support threshold を持つ二つの toy world で、compatibility / path predicate だけが異なる。一方では explicit compatible plan により threshold satisfaction が成り立ち、もう一方では certified upper bound により threshold satisfaction が否定される。

同じ toy は、`RequiredSupportNecessaryForClaim` bridge 付きの claim-license witness も持つ。reachable toy では、explicit compatible plan により toy claim の support-threshold 側が `ClaimLicensedUnderRequiredSupport` として成立する。blocked toy では、同じ required-support bridge shape があっても、support threshold が満たされないため、その toy claim は support layer では license されない。

Lean が固定しているのは、raw support summary や単純な support values だけでは、compatible / feasible な deliverable support claim や bridge-scoped support-layer claim license を同定できない、という狭い非同一化である。

さらに `StructuralPersistenceTemporalResourceLogRatio` は、M側を log-additive ledger に入れる場合の型境界を固定する。ここでは raw な $\log M$ は使わない。M の amount は人員、電力、期限、権限、到達可能性などの単位を持つことがあるため、log readout は $M_{\text{available}}/M_{\text{required}}$ または certified cap / required support のような無次元比に限る。

正の有効資源領域では、available support が required support を下回ると required-normalized support log margin は負になり、符号を反転した signed support log-shortfall は正になる。これは M側の不足を L側の log-additive 帳簿と同じ形式で読むための境界であって、 $M_{\text{available}} = 0$ を log で扱うものではない。ゼロ有効資源は、既存の S-zero / boundary-event 層に残る。

同じファイルは、positive-support regime に限定して、required-normalized な S-potential deficit の加法分解も固定する。すなわち $S = M_{\text{available}} \exp(-L)$ のとき、 $-\log(S/M_{\text{required}})$ は structural loss L と signed support log-shortfall の和として読める。Lean 側ではこの読みを `requiredNormalizedPersistenceDeficitNats` として名づけ、bits で読む場合は `requiredNormalizedPersistenceDeficitBits = requiredNormalizedPersistenceDeficitNats / \log 2` として、単なる底の変換に留める。

`DeficitWeightNonIdentification` は、この加法分解の境界を独立性定理として切り分ける。

対数単位にそろえることは、構造損失と M 側不足の交換レートを決めることではない。DeficitCombiner は結合赤字の公理（台帳合成に対する加法性、各座標断面の連続性、非負入力に対する非負性）を明示し、DeficitCombiner.eq_weightedDeficitFromComponents は、この公理を満たす読み出しが非負重みの線形族 $\alpha L + \beta \ell_M$ にちょうど特徴づけられることを、スカラー表現定理と同じ Cauchy 機構で示す。そのうえで unitWeightDeficitCombiner と structuralHeavyDeficitCombiner は、同じ公理をすべて満たしながら重みの異なる二つのモデルであり、deficitCombiner_axioms_do_not_force_unit_weights がここから $\alpha = \beta = 1$ の非強制を導く。weighted_deficit_intervention_ranking_reverses は、理論編第 8 節（小さな数値例）の数値による厳密有限分離例であり、重み $1:1$ と $2:1$ で介入の赤字順位が反転することを固定する。対になる正の結果として、weightedFullPotential_collapse_boundary_weight_invariant は、 $S = 0$ の崩壊境界が任意の正の重みで不変であることを示す。representation_axioms_do_not_force_k_eq_one は、表現公理だけでは構造側の対数単位 $k = 1$ が強制されないことを、ゼロ損失端点の小さな例で記録する。

この特徴づけは、公理の選択に条件づけられている。線形性を強制しているのは台帳合成に対する加法性の公理であり、 $L \cdot \ell_M$ のような相互作用項、max 型、閾値型の結合は公理段階で排除されている。線形族への限定は、加法的台帳というモデリング選択の帰結である。また、この特徴づけは DeficitCombiner の公理を満たす L 対 ℓ_M の赤字結合層に限る。L_profile の合成層で certificate として許容している kill-switch 型や thresholded-sum 型の合成規則は、成分から L への合成という別層に属し、joint additivity を満たさないため族の外にある。両者は矛盾しない。したがって、 $(1, 1)$ の D_nat は標準候補であり、境界の位置は重みに依らないが、境界内部の順位は重みに依存する。

一方、この恒等式は経験側の threshold hypothesis を定式化するための足場になる。すなわち、domain 側で L 、 $M_{\text{available}}$ 、 M_{required} 、endpoint、cutoff、baseline、no-leakage guard が事前固定されている場合、requiredNormalizedPersistenceDeficitNats が failure / recovery / non-license 境界を held-out で切るかを検査できる。Lean が固定するのは、その検査に入れる量が raw $\log M$ ではなく required-normalized な log-deficit として読まれていることである。

StructuralPersistenceAdmissibilityRepresentation は、これらの部品を入場証明書束として束ねる。ただし入場は二段に分ける。core-entry layer は ontology well-formedness と L_profile composition readout までを要求する。claim-bound layer は、その上に M側 support family、required support、qualified-support boundary、domain-claim bridge、positive log-deficit inputs を追加する。さらに強い監査用の variant は、境界を ProtocolScopedQualifiedSupportFrontierCertificate として要求し、frozen protocol、no-outcome-leakage、provenance guards を境界と一緒に運ぶ。

core-bundle constructor は core prerequisites から core certificate bundle を構

成し、claim-bound bundle constructor は claim-bound prerequisites から claim-bound certificate bundle を構成する。protocol-scoped bundle constructor は、protocol-scoped prerequisites から protocol-scoped certificate bundle を構成する。したがって、構造持続理論で admissible な対象は、使いたい帰結の強さに応じた証明書束を持つ。

StructuralPersistenceFrontierAdequacyHandoff は、この protocol-scoped boundary と empirical adequacy をさらに分ける。ProtocolScopedFrontierAdequacyPending は、boundary が frozen protocol / no-leakage / provenance guards を持つが、empirical adequacy review はまだ別に必要である状態を記録する。FrontierEmpiricalAdequacyCertificate と EmpiricallyAdequateProtocolScopedFrontier は、その別 certificate が供給された場合だけ empirical adequacy support metadata を射影する。

StructuralPersistenceResilienceReadout は、レジリエンスを universal scalar としてではなく、function / carrier / shock / horizon / target に scoped された positive-support readout として扱う。そこでの resilience potential は required-normalized S-potential ratio であり、resilience deficit は既存の requiredNormalizedPersistenceDeficitNats である。したがって、正の有効資源領域では、resilience deficit は after-shock structural loss と M-side support log-shortfall の和として読む。さらに、support boundary から resilience claim predicate へ進む場合も、既存の supplied necessary / sufficient support-threshold bridge を要求する。

StructuralPersistenceResilienceBridgeCertificate は、この readout を claim predicate に使うための tolerance bridge を分ける。ResilienceDeficitTolerance は predeclared / claim-scoped な最大 deficit を記録し、必要条件 bridge がある場合だけ tolerance 超過から claim non-license を、十分条件 bridge がある場合だけ tolerance 以下から claim license を射影する。

StructuralPersistenceClaimBoundaryTightness は、この一方向 bridge を「tight bound」と呼んでよい条件をさらに狭くする。すなわち、同じ threshold / tolerance に対して必要条件 bridge と十分条件 bridge の両方が supplied されている場合だけ、claim predicate と threshold satisfaction、required $\leq$ capacity、または deficit within tolerance の同値を読む。これは熱力学や情報理論でいう「不可能性と達成可能性が同じ壁で閉じる」形への最小 interface である。

StructuralPersistenceClaimBoundaryReport は、この境界を report object としてさらに分ける。necessary-only report は non-license 側だけ、sufficient-only report は license 側だけ、tight report は同値、protocol-scoped tight report は no-leakage / provenance guards 付きの formal 同値、adequacy-pending tight report は empirical adequacy をまだ主張しない formal 同値である。

`StructuralPersistenceClaimBoundaryPublicReport` は、これを外向きの `safe wording` へ渡す層である。`ClaimBoundaryPublicVerdict` は `necessary-only non-license`、`sufficient-only license`、`formal tight equivalence`、`protocol-scoped formal equivalence`、`adequacy-pending formal equivalence` などの表示ラベルを分ける。各 `public report` は `ClaimBoundaryPublicLimitations` を持ち、外向き表示の主張範囲を記録する。

これらの層の証拠ステータスは `docs/lean/THEOREM_STATUS_LEDGER.md` に分けて記録する。`PROOF_AUDIT.md` は宣言単位の監査表であり、`THEOREM_STATUS_LEDGER.m` は `exact accounting identity`、`non-identification / no-go`、`supplied certificate bridge`、`finite toy witness`、`synthetic benchmark support`、`external empirical open` を読者向けに分ける台帳である。

以上をまとめると、M 側の Lean 層は次のような `typed accounting stack` として読むのが安全である。

層	Lean が閉じること	主張境界
M_proj	included temporal claims の有限和と、 $S = M_{\text{proj}} * \exp(-L)$ の符号境界	第7節
support plan / bound	feasible / compatible plan が required support に届くか、または certified upper bound に より届かないこと	第7節
qualified-support boundary	supplied capacity が上界 かつ達成可能である場合 に、 <code>SupportThresholdSatisfied</code> <code>iff required <= capacity</code> を 読むこと	第7節
log-ratio	positive-support regime での $M_{\text{available}} / M_{\text{required}}$ log-shortfall、normalized S deficit の加法分解、 nats/bits readout	第7節

層	Lean が閉じること	主張境界
required-support certificate	M_required を target / horizon / claim / evidence / boundary / necessity に scoped して から narrow threshold へ 射影すること	第7節
claim bridge	required threshold failure を support-layer claim non-license へ接続 すること	第7節
tight-bound bridge	必要条件と十分条件が同 じ threshold / tolerance に対して supplied された 場合だけ、claim と境界条 件の同値を読むこと	第7節
claim-boundary report	one-sided / tight / protocol-scoped / adequacy-pending の状態 を分け、片側 bound を tight と誤読しないこと	第7節
public claim-boundary report	formal status を safe outward-facing wording へ写し、limitations を保 持すること	第7節
finite support / gate toy	raw support summary で は deliverable support / typed mandatory-gate claim license を同定でき ない有限 witness	第7節
workload generated support boundary	有限 workload law と有限 候補集合から、指定リスク 水準を制御する generated required support を構成 し、covered / shortfall readout を分けること	第7節

層	Lean が閉じること	主張境界
workload grid / bracket / limit boundary	finite boundary を supplied real threshold / bracket / refinement / limit sequence に対して近似し、strict side を eventual に読むこと	第7節
monotone risk generated boundary	単調 risk-control predicate から riskFrontier = slnf {capacity riskControlled capacity} を読み、下側非制御と上側制御を certificate-relative に分けること	第7節
positive-rate boundary interpretation	riskFrontier 上側の supplied rate / envelope から overflow probability の 0 収束を読むこと	第7節
resource-conserving protocol class	deliveredSupport <= resourceUse <= availableSupport から scoped interface converse へ渡すこと	第7節

この意味で、M は「便利な特微量」ではなく、M claim を使うための射影、有効資源量閾値、log readout、claim license の型境界として扱われる。

この区別は StructuralPersistenceIdentifiabilityBoundary にも反映されている。MLFormalAccountingStatus は、M と L が同じ accounting interface に載ることと、経験的同格性には別 validation が必要であることを分ける。MLEvidenceStatusBoundary は、L 側の held-out predictive support を M 側へ水増ししたり、M 側の有限 anti-collapse / licensing anchor を外部実ドメインの予測支持として読んだりしないための境界である。

QSA finite benchmark と SRE-H1 finite matched benchmark は、この Lean stack の外側にある経験的 artifact である。これらは、同じまたは統制された raw resource / 単純 support 和 / required support / L / damage 条件を持つ matched 設計で、qualified support gate / path / license だけを変えた readout が primary

/ fresh の両方で positive となり、外部 Windows/Python 環境でも decision が再現された（記録は `externally_rerun_reproduced` および `externally_rerun_reproduced_with_packaging_note`。ステータス：合成ベンチマーク支持＋外部再実行再現性）。これは、Lean が閉じた anti-collapse 型境界を有限 benchmark 側から補強する補助証拠であり、Lean 定理でも、外部実データによる一般の経験的支持でもない。ステータス語彙と読み替え禁止の一覧は理論編に従い、詳細な境界は M側補論に譲る。

これらは、物理法則の再証明ではない。目的は、式の難しさを見せることでもない。むしろ、構造側と資源側を分けて帳簿に載せたあと、その帳簿の中で符号、射影、失敗読みが別々のピースとして正しく流れることを確認している。

4 型境界の次の改修点

今回の理論境界に合わせると、Lean 側で次に明示すべき型境界は三つある。

第一に、LProfile にデフォルトの加法合成を与えないことである。対数損失の加法性は、正の質量軌道における L_mass の性質であり、L_profile 全体の性質ではない。形状、連結性、到達可能性、制御可能性、境界余裕は、単純な足し算で合成できるとは限らない。

すなわち、LProfile にはデフォルトの加法合成を与えず、単一スカラーへの totalization には必ず CompositionRuleCertificate を要求する。

したがって、LProfile から単一の総損失を読む theorem は、CompositionRuleCertificate、たとえば component readout、weighted sum、bottleneck rule、kill-switch rule、lexicographic priority rule、thresholded-sum rule、domain-specific aggregatorなどを明示的に要求すべきである。

この境界は StructuralPersistenceLossProfileCompositionTaxonomy で少し具体化されている。そこでは、L_profile の component readout、weighted sum、bottleneck、kill-switch、lexicographic priority/fallback、thresholded sum を、それぞれ supplied certificate から既存の composition-rule interface へ落とす。重要な点は、これらが「使ってよい合成規則の型」を与えるだけで、どの rule が現実ドメインで妥当か、どの rule が予測に効くか、どの boundary を正しく検出するかは、別途 domain certificate / validation に残ることである。

第二に、BoundaryEventProtocol である。連続 ledger から structural stop、absorbing collapse、regime switch へ移る条件は、結果を見た後で切り替えてよいものではない。境界イベントを使うには、事前固定された trigger predicate と、それがどの evidence で発火したかを示す supplied certificate が必要である。

すなわち、連続 ledger から boundary event layer へ移るには、事前固定された trigger predicate と supplied certificate を経由しなければならない。

第三に、TemporalEffectiveResourceClaim である。有効資源 M は静的な在庫ではなく、target、horizon、assumptions、path dependencies、certificate を持つ条件付き claim として扱うべきである。過去の claim は書き換えず、前提失効、評価損、訂正、実現、期限切れを後続イベントとして追記する。

as-issued view:

発行時点の情報集合で licensed された M claim。

restated view:

現在までの invalidation / impairment / correction を反映した再評価。

Lean 側で狙うべき方向は、過去 claim の mutable status を保存することではない。追記型の ledger event から、asIssuedView と restatedView を純粋関数として導出することである。

5 近似測定をどう扱うか

現実では、真の L 、 B 、 M 、 $m(V_K)$ を直接測れることは少ない。したがって Lean 形式化は、近似を禁止しない。

ここで、対象側の真の境界量と、報告された推定量を分ける。真の $S = M \exp(-L)$ が正しく ground されているなら、 $M = 0$ や collapse boundary による $S = 0$ は、その claim の形式上のゼロ境界である。一方、実務で得られる $\hat{S}$ は、proxy、測定、証明書、近似モデルから作られた報告量である。したがって、 $\hat{S} = 0$ や $\hat{S} \simeq 0$ を真の $S = 0$ と同一視するには、誤差境界、測定プロトコル、bridge certificate が必要である。

ただし、候補の代理指標をそのまま真値として使わない。

真の量 X と代理指標 $\hat{X}$ のあいだに

$$|\hat{X} - X| \leq e_{\text{proxy}}$$

という誤差境界があり、判定の余裕がその誤差より大きいなら、判定は保守的に運べる。Lean は、このような誤差境界と余裕幅から、判定がひっくり返らないことを確認する。

一方で、代理指標が現実の良い近似であるかどうかは、ドメイン側の問題である。評価窓を固定し、事後選択を避け、未使用データで検証し、支持されなかった代理指標も記録する必要がある。

つまり、理論側は近似を受け入れるための形式を与える。ドメイン側は、その近似が妥当である根拠を与える。

6 公式の代理指標は、それだけでは定理に使えない

代理指標の登録簿では、候補、凍結された候補、支持された代理指標、公式の代理指標、支持されなかった代理指標などを区別する。

しかし、公式の代理指標になったこと自体は、数学的な結論を出す根拠ではない。公式という状態は、評価手順、外部再実行、漏洩チェック、記録管理を通ったという制度的な状態である。

定理に使うには、さらに誤差境界、上下界、保守的な健全性条件、余裕幅などが必要である。

したがって、正しい流れは次である。

公式の代理指標 + 誤差・余裕・健全性の根拠 $\implies$ 定理に使える入力

この線を守ることは重要である。そうしなければ、制度的な信頼と数学的な根拠が混ざってしまう。

Lean は、悪い proxy を予測的にするわけではない。Lean が固定するのは、proxy target、誤差境界、claim scope、必要証明書である。同じ ledger、certificates、composition rules、boundary predicates が与えられれば、どの claim が licensed / non-licensed / certificate-required かを再現的に計算できる。しかし、proxy が現実の V_K 、 $L_profile$ 、 M 、boundary event をよく近似しているかは、別途ドメイン側の検証で示す必要がある。

7 混同を防ぐための監査

このリポジトリでは、定理や一部の定義を監査表に分類している。目的は、記帳整理を深い定理のように見せないことだけではない。より重要なのは、どの宣言がどのピースを扱っているのかを明示し、構造側、資源側、報告ラベル、証明済み readout、代理指標、回復判定を混ぜないことである。

分類は、おおむね次のようになる。

- 実質的な接続: 異なる読み取り層を本当に接続する。
- 根拠の露出: 与えられた witness や certificate を取り出す。
- 非同一性: 二つの読みが同じではないことを示す。
- 記帳整理: データを束ねる、射影する、名前を整える。
- リスク: 過大主張や循環の危険がある。

Lean は証明が通ることを確認する。しかし、その宣言がどの読み取りピースを扱い、どの混同を防いでいるかは自動で読者に伝わらない。だから、別に監査表が必要になる。

8 何を保証し、何を保証しないか

Lean が保証するのは、与えられた入力からの境界帰結である。

たとえば、正の質量軌道が与えられれば、対数損失読み取りが整合することを確認する。構造損失と有効資源が与えられれば、境界ポテンシャルの符号が正しく流れることを確認する。誤差境界と余裕幅が与えられれば、代理指標による判定が保守的に保存されることを確認する。共有資源と必要量が与えられれば、個別には可能でも同時には不可能な場合を確認する。

ここでの硬さは、主に恒等式、型分離、証明書境界の硬さである。Lean は、対数損失の telescoping、 S の符号の流れ、TemporalEffectiveResourceClaim の追記型評価、 $L_profile$ の合成規則要求、boundary protocol の事前固定といった、入力後の推論を守る。

一方で、Lean は次を保証しない。以降の本文に散っていた防御文は、この一覧に集約する。

- 現実の測定が正しいこと。
- $F/K/V_K/m$ の選び方が最善であること。
- 各ドメインの固有理論が正しいこと。
- 代理指標の誤差境界が現実妥当であること。
- どの対象を優先すべきかという価値判断。
- 世界の失敗様式がこの分類で尽くされること。
- capacity の探索、optimizer、max-flow/min-cut、path discovery。
- CanRecoverTo、実回復、実生存、回復不能性、生存不能性。
- support threshold の認可を、domain claim 全体の認可へ自動昇格すること。

- M 側の empirical support、required amount の経験的正しさ、実ドメインでの有効資源量測定。
- tight bound interface を、境界量の自動発見または universal law として読むこと。
- protocol-scoped boundary を、empirical adequacy や測定妥当性として読むこと。
- 一般 queueing theorem、LDP、effective bandwidth theorem、Shannon theorem、任意 protocol converse。
- cross-domain predictive transfer、介入効果、実回復、実生存、実ドメインの測定妥当性。

この境界は、本文の最後だけでなく、形式化の設計そのものに入っている。強い結論は、強い根拠があるときだけ出る。

Lean アーティファクトは、ドメイン側の bridge が弱い場合には、その弱さを隠さず、claim を certificate-required、unlicensed、または provisional に留めるための装置である。より強い law-like result は、ドメインごとの attainability / impossibility bound が入ったときにだけ昇格する。

9 接続例

形式化には、有限 Ising 型モデル、構造診断ポリシー、代理指標の登録簿、横断報告などの接続例がある。

有限 Ising 型モデルでは、スピン配置、磁化に関する述語、維持可能領域を明示できる。このとき Lean は、供給された有限の根拠の範囲で、維持、非維持、資源側読みを分けて扱う。

まず、一つの対象だけでも、構造側の制約 L/B と有効資源 M を分けて読むことには価値がある。Lean 側では、その単体読みを、維持、非維持、資源側崩壊、近似測定、回復判定などの別々のデータと定理として扱う。

その上で、横断報告では、複数ドメインの構造側・資源側の読みを並べることができる。共通なのは、持続・崩壊・回復・外部性を読むための境界形式である。

この共通形式があることで、別ドメインの事例も「同じ現象」としてではなく、「同じ読み取り座標を動かす介入」として比較できる。資源を増やす介入、名目資源を有効資源へ変換する介入、構造側の制約を下げる介入、結合を切って外部性を減らす介入は、ドメインが違っても同じ種類の読みを持つ。

この意味で Lean が扱う transfer は、形式的・スキーマ上の transfer である。

すなわち、異なるドメインの対象を、供給された adapter と証明書のもとで、同じ $L/M/S$ 型、同じ claim-license 型、同じ report handoff へ載せられる、という主張である。これは、二つのドメインの native mechanism が同じであることや、一方で学んだ介入規則が他方の outcome を予測することを含まない。

経験的・予測的 transfer は別問題である。あるドメインで得た $+M$ 、 $-L$ 、boundary monitoring、repair-path intervention などの介入タイピングが別ドメインでも効くかどうかは、ドメイン側の proxy、誤差境界、bridge certificate、held-out validation によって検証される。形式的な cross-domain transport certificate があっても、それだけでは target domain の outcome prediction は license されない。Lean は、その検証済み入力を与えられた後の境界帰結を固定する。検証済みでない経験的一般化を theorem として生成しない。

Lean 側で守るべきなのは、この横断性を過大主張にしないことである。横断報告は、個別の読みを一つに潰さず、どの対象の L/B が変わり、どの対象の M が変わり、どの集合が同時に回復できるかを別々に持つ。これにより、個別最適と全体最適のずれや、修復外部性の位置を境界帰結として扱える。

特に、多資源の結合では、各資源を別々に見た射影では回復可能に見えても、同じ選択で全資源を同時に満たすと不可能になる場合がある。Lean 側では、このような choice-coupling 型の失敗を、単なる一資源の予算超過とは別の読みとして扱う。

10 結論

構造持続理論の Lean 形式化は、証明書つき入力から、どの境界帰結が出るかを検証する装置である。

その価値は、構造側の制約 L/B と有効資源 M の分離を、実装の中でも崩さないことにある。

この分離が崩れると、機能停止、資源側崩壊、回復不能、名目資源の変換不能、集合的同時回復不能、修復外部性が、どの原因に由来するのか見えなくなる。Lean は、その混同を防ぐための形式的なガードレールである。

したがって、この Lean 形式化は、型付き境界インターフェースとして使うときに、入力された測定、代理指標、証明書から、どの帰結を出してよいかを狭く固定するための実装である。保証しない範囲は第7節に集約した。

11 付録 A. 代表的な Lean 宣言

11.1 A.1 構造損失カーネル

- RepresentationTheorem.loss_must_be_log
- TelescopingExp.measure_eq_initial_mul_exp_neg_cumulative_loss
- GeneralStateDynamics.feasibleMass_eq_initial_mul_exp_neg_cumulativeNetAction
- StructuralPersistenceBalancePrinciple.pathwise_netConsumption_exponential_kernel
- StructuralPersistenceBalancePrinciple.local_exponential_netConsumption_identity

11.2 A.2 L/B と M の分離

- StructuralPersistenceOntology.inherits_commonPersistenceProfile
- StructuralPersistenceOntology.fullPotential_collapse_iff_effectiveResource_nonpos
- StructuralPersistenceOntology.fullPotential_pos_iff_effectiveResource_pos
- resourceCollapseAt_and_state_mem_to_maintained_and_fullPotentialCollapse
- structuralStopAt_and_effectiveResource_pos_to_fullPotential_pos
- ineffectiveRawResourceAt_unwrap_bottleneck
- ComputedTemporalResourceProjectionReport.projectedEffectiveAmount_eq_sum
- ComputedTemporalResourceProjectionReport.not_mem_included_of_recorded_invalidation
- TemporalResourceSPotentialProjection.sPotential_eq_zero_of_projectedAmount_eq_zero
- TemporalResourceSPotentialProjection.sPotential_pos_iff_projectedAmount_pos
- TemporalResourceSPotentialProjection.sPotential_nonpos_iff_projectedAmount_nonpos
- TemporalResourceSPotentialProjection.sPotential_mono_projectedAmount_same_loss

11.3 A.3 入場管理と構造オントロジー

- AdmissionReady
- AdmissionFailure
- AdmittedOntology
- AdmittedExactOntology
- StructuralPersistenceWellFormed
- StructuralPersistenceExactViableRegion
- StructuralPersistenceViableMassCertificate
- StructuralPersistenceMassModelCertificate

11.4 A.4 実行時判定と回復境界

- `activatedRuntimeFailure_has_classification`
- `classification_nonempty_iff_activatedRuntimeFailure`
- `outsideViableRegionAt_to_not_maintained`
- `resourceCollapseAt_to_fullPotentialCollapse`
- `structuralStopAt_of_certificate`
- `recoveryShortfallAt_to_notCanRecover`

11.5 A.5 代理指標と登録簿

- `ApproxLossProxy.abs_error_bound`
- `ApproxResourceProxy.abs_error_bound`
- `ApproxNetBurdenProxy.abs_error_bound`
- `ApproxLossProxy.failure_of_proxy_margin_and_true_sound`
- `ApproxNetBurdenProxy.failure_of_proxy_margin_and_true_sound`
- `ProxyValidationCertificate.no_post_hoc_selection`
- `NoSupportRecord.lacks_empirical_support`

11.6 A.6 診断・横断報告・接続例

- `admittedExactMLSeparationDiagnosticSplitInterface`
- `structuralDiagnosticPolicyRecoverable_of_required_sum_le_budget_and_attainable_le`
- `structuralDiagnosticPolicyBlocked_of_required_sum_exceeds_budget`
- `structuralDiagnosticForbiddenPolicy_of_required_interval`
- `structuralDiagnosticFrontierReport_of_certificates`

11.7 A.7 workload boundary / generated riskFrontier

- `FiniteStochasticWorkloadGeneratedOperationalFrontierTheorem.generatedRequiredSupport_controls_risk`
- `FiniteStochasticWorkloadGeneratedOperationalFrontierTheorem.availableRiskControlled_of_generatedRequired_le_available`
- `FiniteStochasticWorkloadGridQuantileApproximationFrontier.idealThreshold_le_gridQuantileSupport`
- `FiniteStochasticWorkloadGridQuantileApproximationFrontier.gridQuantileSupport_le_upperBound`
- `FiniteStochasticWorkloadRealThresholdBracket.gridRequiredSupport_sandwich`
- `FiniteStochasticWorkloadBracketRefinementReadout.fineGridRequiredSupport_le_coarseGridRequiredSupport`
- `GeneratedRiskFrontierTerminology.not_riskControlled_of_capacity_lt_generatedRiskFrontier`
- `GeneratedRiskFrontierTerminology.riskControlled_of_generatedRiskFrontier_lt_capacity`

- GeneratedRiskFrontierTerminology.generatedRiskFrontier_eq_sInf_controlledSet
- GeneratedRiskFrontierTerminology.eventually_baseRiskControlledAtAvailable_of_generatedRiskFrontier_It_available
- ResourceConservingProtocolClass.protocolObstruction_and_positiveD_of_available_It_required